6-Week C++ Course: Systems-Oriented Track

A practical syllabus designed to prepare for Computer Architecture, Operating Systems, Systems Programming, Distributed Systems, and CUDA.

Goal: By the end, you should be able to design and implement a non-trivial C++ program, reason about memory and ownership, write concurrent code, build/debug/test it, and organize it as maintainable software.

Week 1 — C++ Fundamentals

• Compilation vs linking; primitive types; control flow; functions; references; pointers; arrays; strings; structs/classes; const; stack vs heap; basic I/O; namespaces; header/source separation.

• Project: CLI file/text utility. Read and process files, compute statistics, write output, and organize the program across multiple .hpp/.cpp files.

Week 2 — Memory, Ownership & Object Lifetime

• Constructors/destructors; RAII; ownership; dynamic allocation; unique_ptr/shared_ptr/weak_ptr; copy constructor/assignment; move constructor/assignment; lvalues/rvalues; std::move; rule of 0/3/5; operator overloading.

• Project: RAII-managed Buffer/Resource Library. Implement allocation, resizing, copying, moving, and automatic cleanup.

Week 3 — STL & Generic Programming

• vector, array, deque, list (conceptual), map, unordered_map, set, queue, stack; iterators; algorithms; lambdas; function objects; optional; variant; tuple; string_view; basic templates; container complexity.

• Project: In-memory key-value store with SET/GET/DELETE/KEYS/SIZE, persistence, parsing, error handling, and tests.

Week 4 — Concurrency

• std::thread; mutexes; lock_guard; unique_lock; condition variables; futures/promises; atomics; race conditions; deadlocks; critical sections; producer-consumer; thread-safe APIs.

• Project: Thread Pool with worker threads, task queue, synchronization, and clean shutdown.

Week 5 — Building Real C++ Software

• CMake; targets; libraries; dependencies; Debug/Release builds; preprocessing/compilation/object files/linking; static/shared libraries; LLDB/GDB; stack traces; memory inspection; unit/integration testing; assertions; benchmarking; profiling.

• Software design: separation of concerns, encapsulation, abstraction, composition over inheritance, dependency inversion, interface design, RAII, single responsibility, avoiding unnecessary abstractions.

Week 6 — Capstone: Concurrent Data Processing Engine

• Combine C++, STL, RAII, CMake, concurrency, synchronization, error handling, logging, tests, benchmarks, and documentation.

• Pipeline: input files → file reader → task queue → thread pool → workers → aggregation → output. Benchmark 1/2/4/8/16 threads and investigate why scaling eventually stops.

Recommended Project Structure

```
my_project/
■■■ CMakeLists.txt
■■■ README.md
■■■ include/my_project/
■ ■■■ processor.hpp
■ ■■■ thread_pool.hpp
■ ■■■ parser.hpp
■■■ src/
■ ■■■ processor.cpp
■ ■■■ thread_pool.cpp
■ ■■■ parser.cpp
■■■ tests/
■■■ examples/
■■■ benchmarks/
■■■ app/
■■■ main.cpp
```

Stopping Criterion

Do not try to finish C++. Move on when you can comfortably write a multithreaded C++ program, manage memory safely, use standard containers, organize a multi-file project with CMake, and compile/debug/test it confidently. You do not need advanced template metaprogramming, obscure STL features, or every modern C++ feature.

How This Connects to the Systems Roadmap

C++ → Memory → Computer Architecture
C++ → Threads → Operating Systems
C++ → Concurrency → Distributed Systems
C++ → Performance → CUDA / GPU Programming

Project Progression

Stage	Project	Primary learning
1	CLI File/Text Utility	Language + compilation
2	RAII Buffer Library	Memory + lifetime
3	Key-Value Store	STL + API design
4	Thread Pool	Concurrency
5	Concurrent Data Processing Engine	Integration + performance

Working rule: Use the internet/books/documentation for learning material, but implement the projects yourself. The objective is to move from “I know C++ syntax” to “I can design and implement a non-trivial C++ system from an empty directory.”